



© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2023

M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_4

4. Browser Extension Architecture

Matt Frisbie¹

(1) California, CA, USA

Understanding how the various elements of browser extensions work is important, but even more important is understanding how they work together. The architecture of browser extensions is unusual and multifaceted:

- They lack centralization and instead are formed by a network of distributed elements: background scripts, content scripts, popup and options pages, and devtools pages.
- They can manage the complex browser page infrastructure that spans across multiple windows and tabs. Because content scripts can run on multiple pages and are independently loaded, they require special considerations such as a multiplexed communication channel and update version control.
- They can offer a web page–based user interface in the popup page, options pages, devtools pages, and content scripts. They can also directly integrate with the native browser chrome with features such as keyboard shortcuts, omniboxes, and desktop notifications.

Note This chapter is a high-level overview covering how the pieces connect and interact. The APIs and other specific details are primarily covered later in this book.

Architecture Overview

The browser extension architecture can be more easily understood visually. Consider the following diagram displaying how all the pieces of browser

extensions fit together (Figure 4-1).

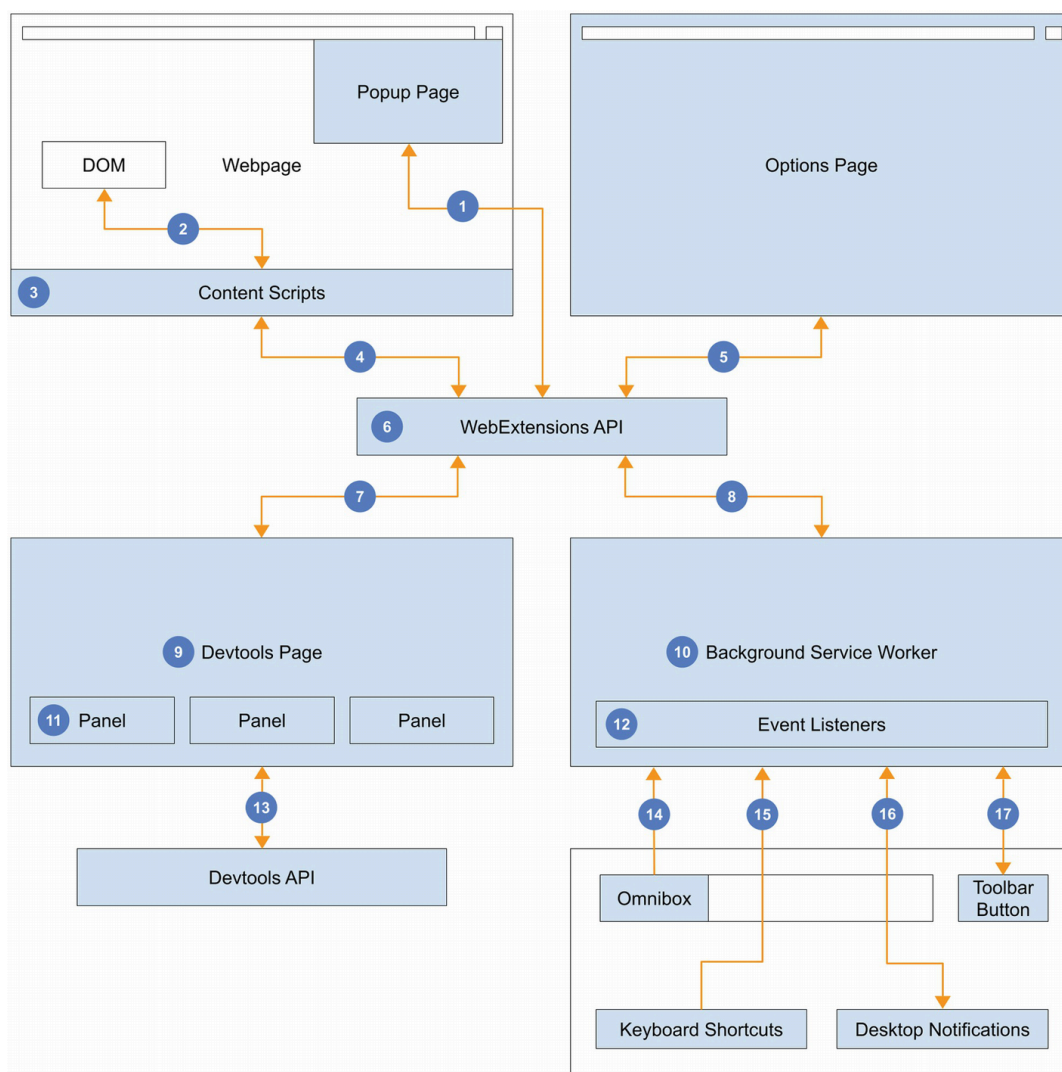


Figure 4-1 Browser extension architecture diagram

Let's unpack the various elements and connections displayed here, identified in the above diagram by number:

1. Popup pages can use the WebExtensions API to perform tasks like reading/writing data to the storage API, sending messages to the background page, or triggering behavior in the active tab's content script.
2. Content scripts can view and manipulate the host web page DOM as well as listen for DOM events.
3. Content scripts are injected directly into web pages and executed in a sandboxed JavaScript runtime. Each tab with a permitted scheme (`http://`, `https://`, `ftp://`, `file:///`) is eligible to have content scripts injected.

4. Content scripts can use WebExtensions API to perform tasks like sending and receiving instructions from the background script, as well as exporting information from the page DOM. Content scripts on a single tab or subset of tabs can be targeted via `chrome.tabs.query()`.
5. In a similar fashion to popup pages, options pages interact with the rest of the extension via the WebExtensions API.
6. The WebExtensions API is the connective tissue for any web extension. It allows for bidirectional messaging between any two elements, as well as access to a shared storage API. The extensions messaging protocol is a broadcast format, meaning that any other part of the extension can listen to all messages being sent.
7. Devtools pages are only able to use a limited subset of the WebExtensions API.
8. Background service workers can manage the rest of the extension via the WebExtensions API. This includes dispatching messages to another part of the extension when an API event handler is triggered.
9. The devtools page is initialized each time the developer tools interface is opened, and torn down when it is closed. It is primarily used to initialize child elements such as panels. Devtools pages are headless web pages.
10. The background service worker is the nerve center of the extension. It is frequently used to handle events, dispatch messages, and perform authentication duties. The background service worker has the useful property of being a singleton; for any number of tabs or windows, only one service worker will ever be running.
11. Using the Devtools API, a devtools page can generate multiple child pages that render natively inside the browser's devtools interface. These child pages take the form of panels and sidebars.
12. The background service worker is the only element of a browser extension that can reliably handle browser events. Elements of

extensions like options pages, popup pages, content scripts, and devtools pages are transient and therefore may miss events when they are not running.

13. Devtools pages are granted access to a supplementary Devtools API which enables methods for creating child views, and inspecting and debugging web pages.
14. The Omnibox can be enabled and configured in the manifest. When a special keyword is typed into the URL bar, a special search-like interface displays. This interface will dispatch omnibox events with the search bar's contents to the extension, thereby allowing the extension to offer search engine-style behavior.
15. Keyboard shortcuts can be enabled to either trigger native behavior like opening the popup page, or to trigger a custom command event.
16. Native desktop notifications can be displayed programmatically. The notifications will also trigger click and close events.
17. The native toolbar icon can perform in one of two ways: it can trigger the popup page, or it can fire an extension click event.

Plurality, Lifecycles, and Updates

Some critical considerations when developing browser extensions are how many of each element can exist at any given time, the fashion in which these elements are created and destroyed, and how they handle updates to the extension.

Background Service Worker

The background service worker is the only extension element that guarantees singleton behavior irrespective of the number of extension pages, windows, or tabs open. The service worker is torn down when the browser detects the service worker is idle and restarted on demand (e.g., when the browser detects an incoming event with a han-

dlr). The browser will restart the service worker when the extension is updated.

NoteFor more details on how background service workers behave, refer to the *Background Scripts* chapter.

Popup and Options Pages

Browsers will ensure that only one popup page is ever opened per window. However, if multiple windows are open, each window can independently open a popup. Popup pages are transient: they initialize when the popup interface expands and torn down immediately when the popup interface closes. The browser will forcibly close all open popups when the extension is updated.

There is no limit to the number of options pages that are opened. However, if you set the manifest option `options_ui.open_in_tab` to `false`, the browser will ensure that only one modal options page is open per window. Options pages, in both modal and tab forms, have a normal web page lifecycle. The browser will forcibly close all open options pages when the extension is updated.

NoteFor more details on how popup and options pages behave, refer to the *Popup and Options Pages* chapter.

Devtools Pages

The devtools page is rendered exactly once each time the browser's devtools interface is opened. Therefore, because there can be only one developer tools interface per window, there can be exactly one devtools page per window. This page and the child pages it creates, consisting of panels and sidebars, will persist as long as the developer tools interface is opened. Importantly, these developer tools pages will not be affected when the extension is updated and so they are susceptible to becoming stale.

NoteFor more details on how devtools pages behave, refer to the *Devtools Pages* chapter.

Content Scripts

Content scripts will be injected into web pages in a manner defined by the extension manifest. Furthermore, multiple content scripts can be injected into a single web page. For an extension with M content scripts defined, and a browser with N tabs open, the total number of content scripts running at any given time is bounded by $M \times N$. Content scripts can be injected into the page upon different pageload events, but all of these are approximately when the page is initially loaded. They behave identically to regular web page scripts and will execute in the same fashion. Importantly, content scripts will not be affected when the extension is updated and so they are susceptible to becoming stale.

NoteFor more details on how content scripts behave, refer to the *Content Scripts* chapter.

Browser Extension File Server

When an extension is installed, the browser will enable the extension's files to be accessible via a simple file server. To explore this, let's use a simple extension that can be used to demonstrate some extension file server concepts. The file structure is as follows:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "content_scripts": [
```

```
{
  "matches": ["<all_urls>"],
  "js": ["content-script.js"]
},
"web_accessible_resources": [
  {
    "resources": ["fetch-page.js"],
    "matches": ["<all_urls>"]
  }
]
}
```

Example 4-1a manifest.json

```
import "../fetch-page.js";
console.log("background.js");
```

Example 4-1b background.js

```
console.log("content-script.js");
import(chrome.runtime.getURL("fetch-page.js"));
const el = document.createElement("script");
el.src = chrome.runtime.getURL("fetch-page.js");
document.body.appendChild(el);
```

Example 4-1c content-script.js

```
console.log("fetch-page.js");
fetch(chrome.runtime.getURL("extra.html"));
```

Example 4-1d fetch-page.js

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Extra Page</h1>
    <script src="fetch-page.js"></script>
  </body>
</html>
```


Example 4-1e extra.html

Take a minute to examine what this extension is doing. Some things to note

- The extension will inject `content-script.js` on all web pages.
- The manifest designates `fetch-page.js` as a web accessible resource and is accessible by all web pages.
- `content-script.js` attempts to load `fetch-page.js` in two different ways: via dynamic `import`, and via dynamic `<script>` loading.
- `fetch-page.js` sends a network request for the `extra.html` file. This file will be loaded and executed in multiple locations in the extension, with different results.

To understand how the extension file server works, we'll unpack this extension one file at a time. Begin by loading the extension in Google Chrome and inspecting the background service worker console output by clicking the *service worker* link in the extension card (Figure 4-2).

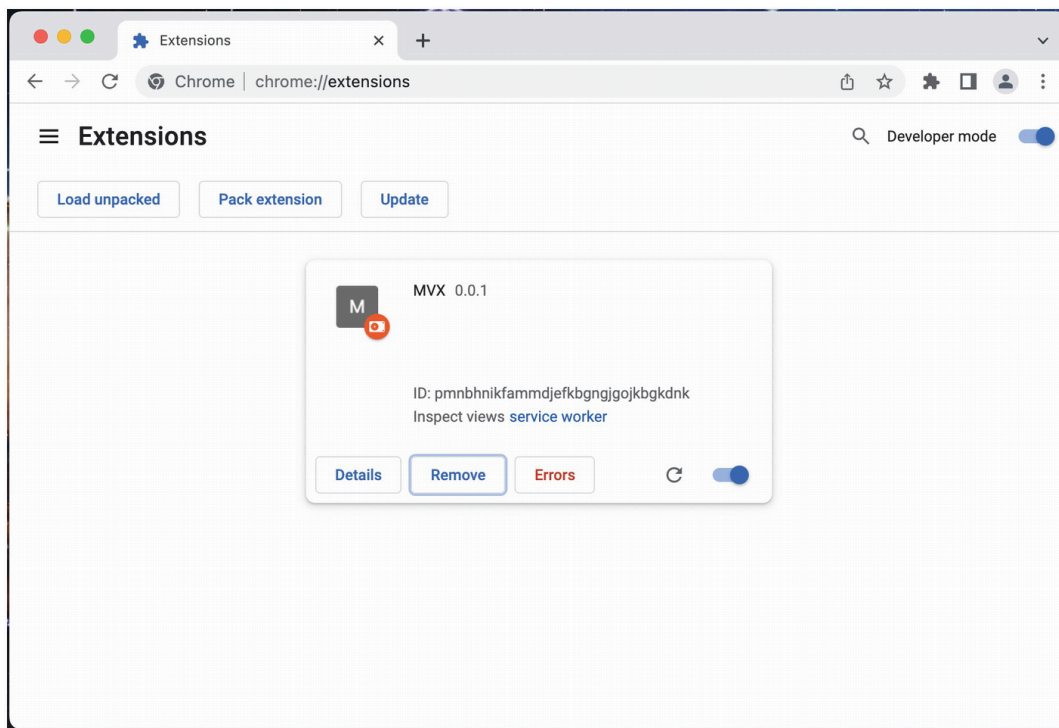


Figure 4-2 Clicking the service worker link will open the background console output

The background service worker console output will appear as in Figure 4-3.

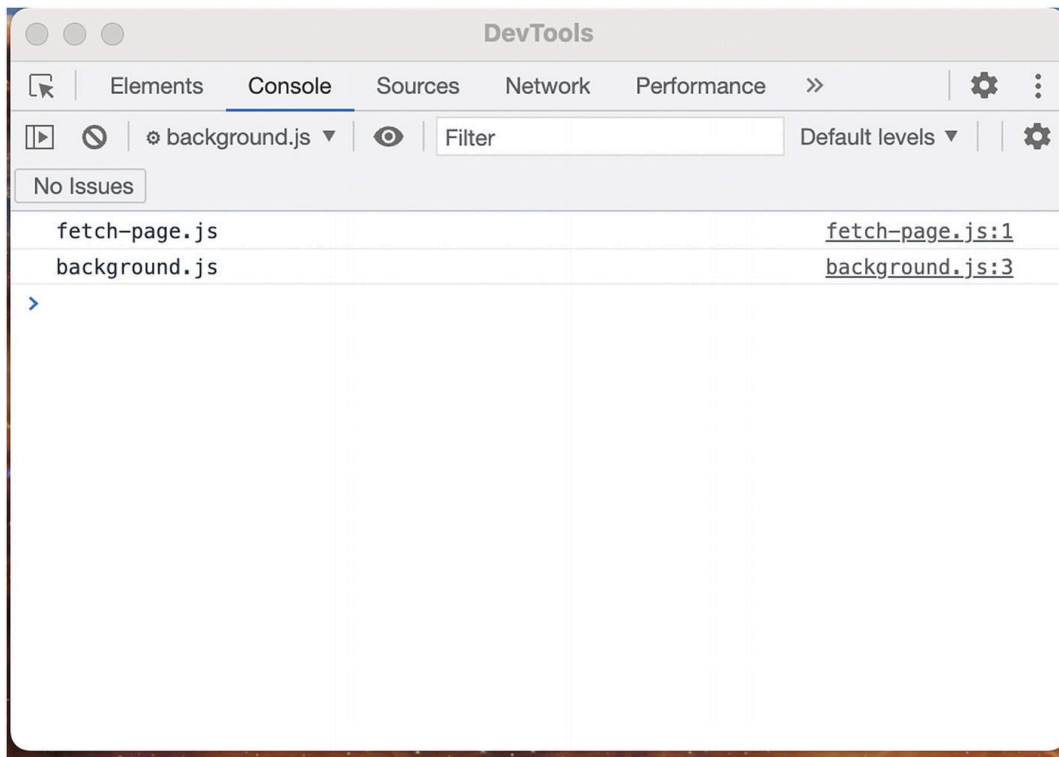


Figure 4-3 The background console output

This indicates that the background script was able to successfully import and execute the `fetch-page.js` script. Let's look to the *Network* tab to see how this occurred (Figure [4-4](#)).

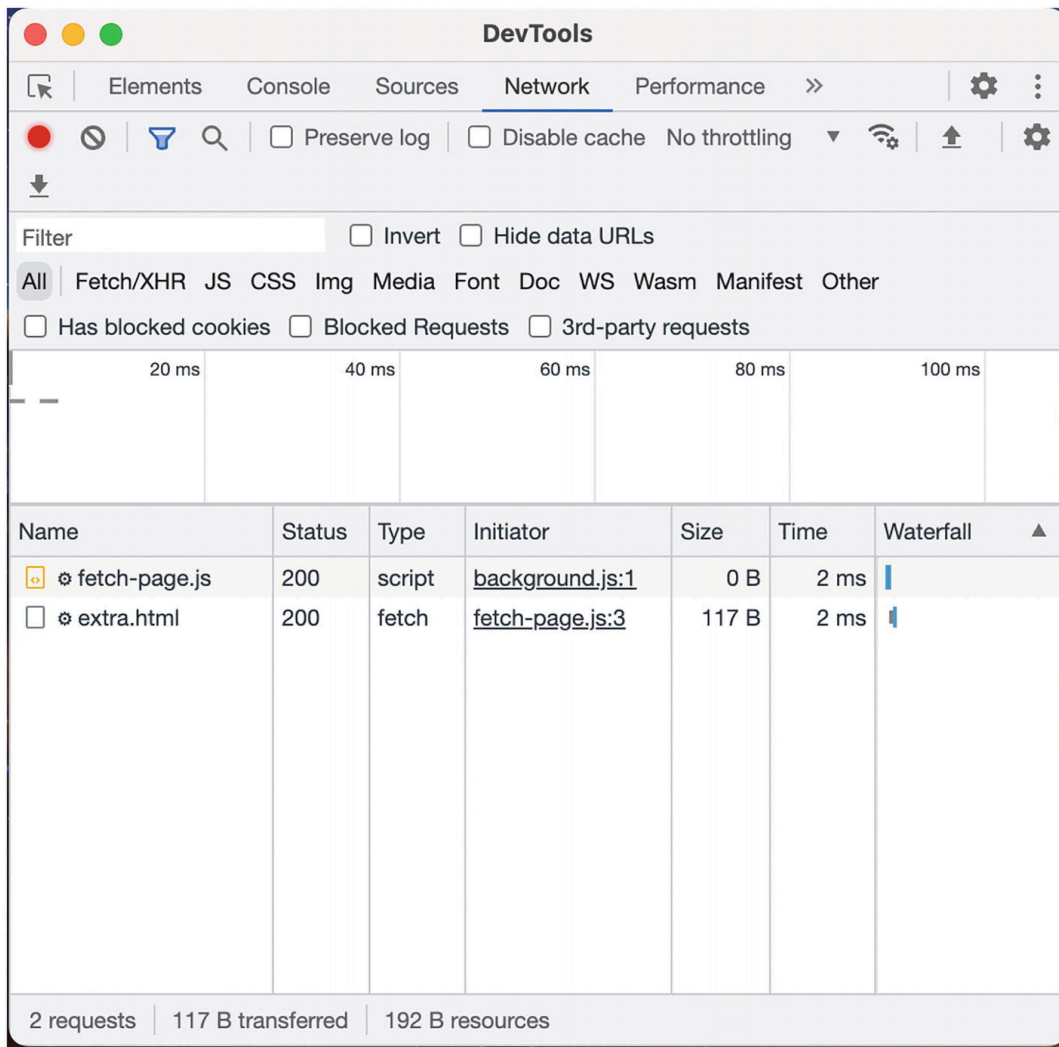


Figure 4-4 The background network activity log

Reload the extension to trigger these network requests. You will see two successful network requests: one for the imported JS file and one for the fetched HTML file. Inspecting the JS request reveals the following (Figure 4-5).

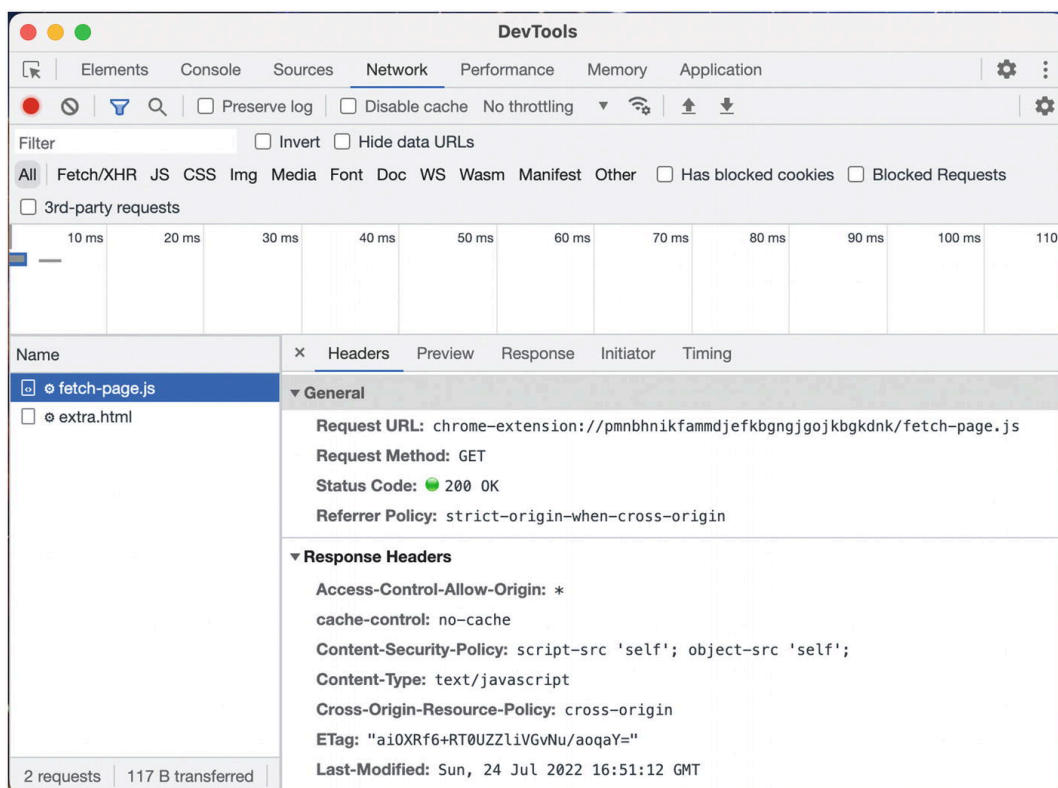


Figure 4-5 The `fetch-page.js` request

This network request successfully sends a GET request to the extension file server, and the file server returns a 200. Note the URL beginning with the `chrome-extension://` protocol. Let's unpack the parts of this URL:

- **chrome-extension://** is the URL protocol, which indicates to Google Chrome that the request should be routed to the browser's installed extensions. The protocol differs between browsers: Mozilla Firefox uses `moz-extension://`, Microsoft Edge uses `extension://`, Opera uses `opera://`.
- **pmbhnikfammdjefkbgngjgojkbkgkdnk** is the extension ID. Your extension's ID will be different. This ID is used to uniquely identify this instance of the extension both inside the browser as well as when published in the extension marketplace. The ID format will vary slightly between browsers: for example, Google Chrome uses an unbroken string of lowercase letters, whereas Mozilla Firefox uses a v4 UUID.
- **fetch-page.js** is the URL path. This will exactly match the file path inside the extension directory.

Next, examine the `extra.html` request (Figure [4-6](#)).

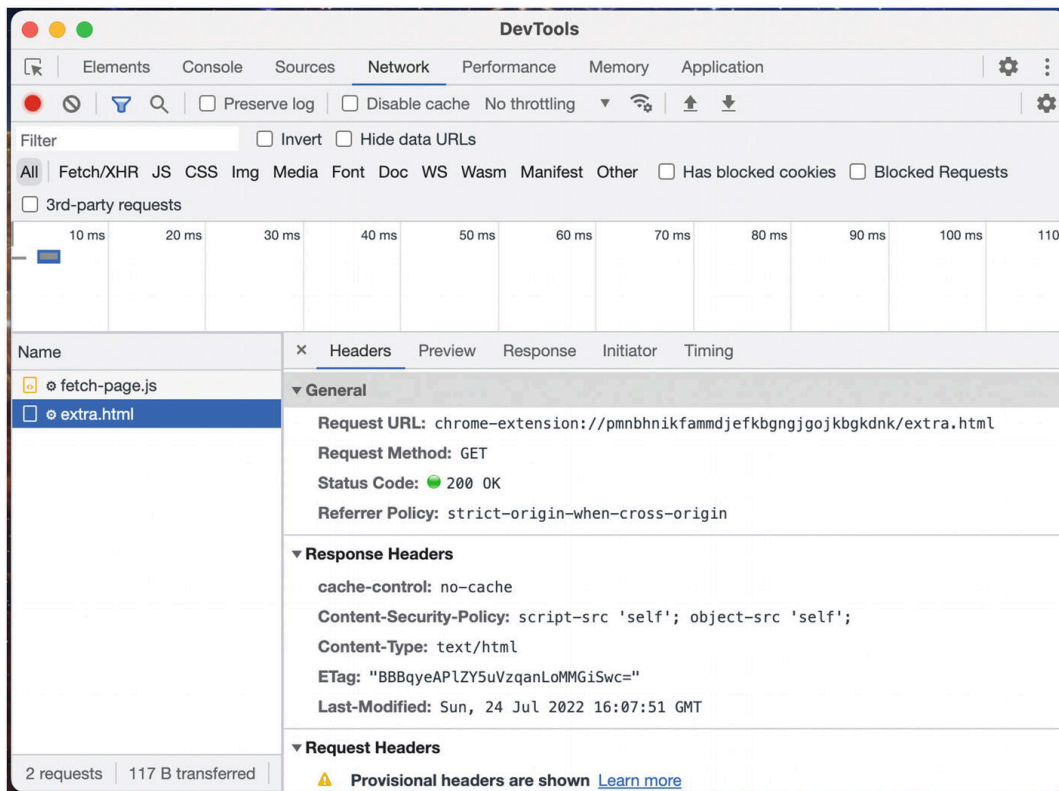


Figure 4-6 The `extra.html` request

Compare the two requests and note the presence or absence of the `Access-Control-Allow-Origin` and `Cross-Origin-Resource-Policy` headers. These are automatically added because `fetch-page.js` is listed under `web_accessible_resources`. Compare this to the request for `extra.html`: you will notice these headers are absent, as `extra.html` is not listed as a web accessible resource.

Note You also will notice that `extra.html` is not referenced directly or indirectly anywhere in the manifest, and yet the extension file server is perfectly happy to return the file. When sending requests from an extension context (background, popup, extension protocol), requests for *any* file in the extension will return that file successfully.

Next, let's copy the URL of the `extra.html` request (yours will differ from the screenshot above) and open it in a new browser tab. Open the developer tools once the page is loaded (Figure [4-7](#)).

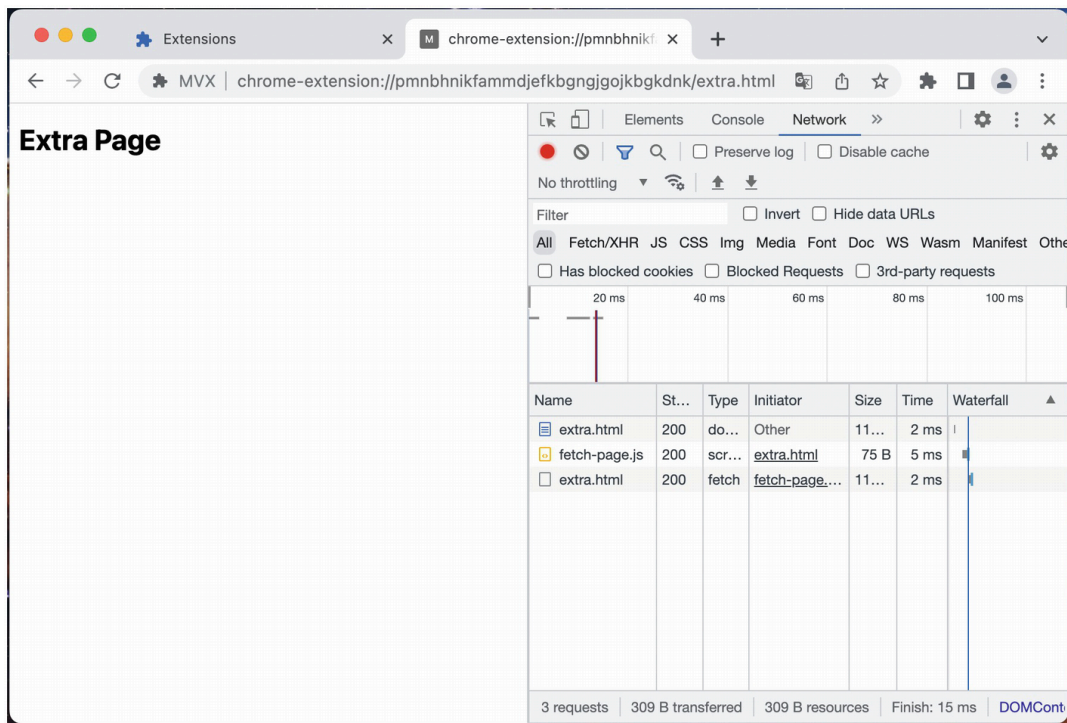


Figure 4-7 The `extra.html` page with network requests displayed

Tip An easy way of accomplishing this is to right click on the network request and select “Open in new tab.”

The browser is loading `extra.html` from the extension file server, rendering it as a normal web page, and successfully loading the `<script>` and executing its payload (which performs a duplicate fetch of `extra.html`). There are some critical takeaways to note here:

- Because it can load and render any file included in the extension, browser extensions can have an unlimited number of web pages, including ones that are not explicitly referenced by the manifest.
- The special `chrome-extension://` protocol routes requests to the file server. Pages rendered in this way can use the WebExtensions API in their scripts, as well as send requests for any file in the extension directory.

This behavior is drastically different when compared to content script execution. Note that the manifest is configured to inject the content script on any web page, so next direct your browser to an inert test site such as `blank.org` and view the console output (Figure 4-8).

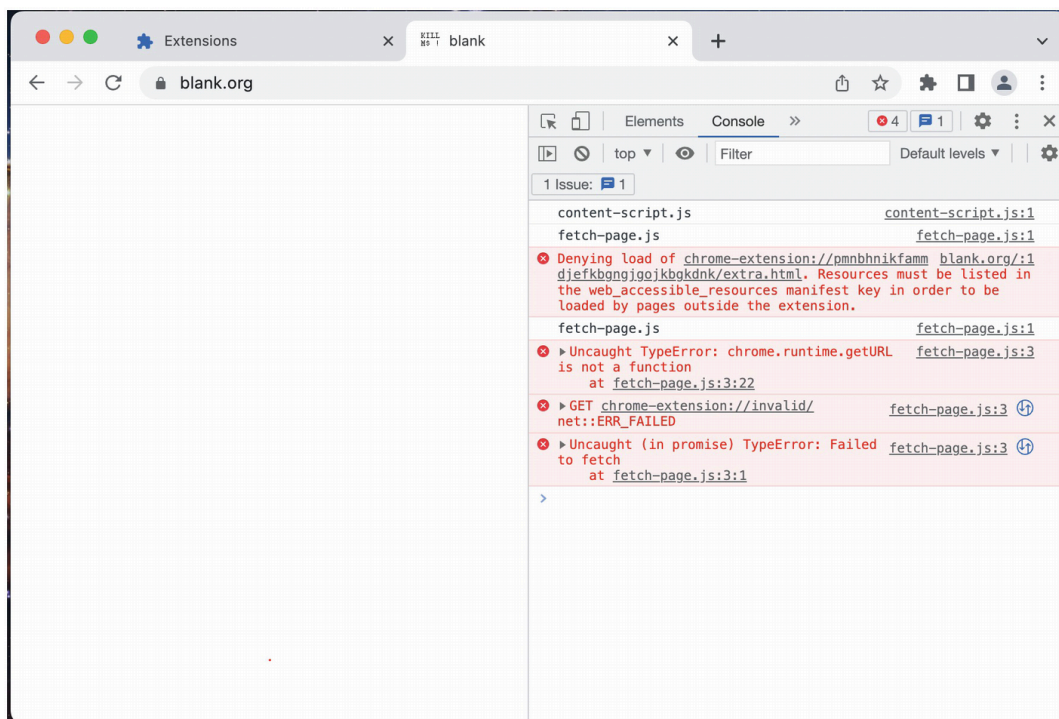


Figure 4-8 The console output from the extension's content scripts

First off, note that the content script is *not* being served via network request. Rather than being loaded from the extension file server, the browser is injecting it into the page directly.

Observe that the content script is attempting to fetch the JS file in two different ways, and each throws an error for a different reason:

- The first attempt dynamically imports the `fetch-page.js` file and executes it. The dynamic import succeeds because `fetch-page.js` is a web accessible resource. However, because this is executing inside a content script, it cannot load the HTML file since it is *not* made accessible under `web_accessible_resources`. This throws a `Denying load error`.
- The second attempt creates a `<script>` element to load the `fetch-page.js` file and execute it. The script load succeeds because `fetch-page.js` is a web accessible resource. However, the execution context of the loaded script is not granted access to the WebExtensions API, and therefore the attempt to use `chrome.runtime.getURL` throws a `TypeError`.

Although content scripts still can access the extension file server, its access is considerably more restricted.

Tip The extension file server is a flexible way to make your extension behave more like a web page with multiple views and routing, but the request routing does not support basic features like redirects or custom 404 pages.

Sandboxed Pages

In manifest v3, the values allowed in the Content Security Policy (CSP) is considerably more limited when compared to v2. It newly disallows the following in extension pages:

- Inline scripts
- Remotely loaded code
- `eval()`
- User-provided scripts

To use these tools, you must run an extension page in a sandbox. Doing so will allow the page to use the disallowed features listed above, but at a cost – the page will lose all access to the WebExtensions APIs.

Note Unless specified, the default sandbox content security policy is

```
sandbox allow-scripts allow-forms allow-popups allow-  
modals; script-src 'self' 'unsafe-inline' 'unsafe-eval';  
child-src 'self';
```

The following example defines the popup page as a sandboxed page. When opening the popup page, the inline script will execute the `eval()` without issue:

```
{  
  "name": "MVX",  
  "version": "0.0.1",  
  "manifest_version": 3,
```



```
"action": {
  "default_popup": "popup.html"
},
"sandbox": {
  "pages": ["popup.html"]
}
}
```

Example 4-2a manifest.json

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Popup</h1>
    <script>
      eval(`document.body.innerHTML += '<div>Foobar</div>'`);
    </script>
  </body>
</html>
```

Example 4-2b popup.html

Summary

In this chapter, you were presented with an overview of how all the elements of a browser extension work in concert. With this knowledge, you should now be able to analyze how a given extension moves information around and makes API calls. This chapter also equips you to more effectively be able to plan for how to take an idea for a browser extension and turn it into actual code. You should also have a good understanding of how the browser extension file server can deliver files in various ways, as well as the tradeoffs involved when declaring a sandboxed page.

The next chapter will cover all the fields that can appear in an extension manifest as well as how each field's definition can control the behavior of an extension.

